

# Égua Mano! Gamer — Manual Prático de Engenharia em Go

**Construção Passo a Passo do Zero:** Game Loop, Física Clássica, Polimorfismo, Áudio Procedural e WASM

**Autor & Engenheiro:** Lucivaldo Junior (Luci Junior) | **Stack:** Go (Golang 1.22+) & Ebitengine v2

## Manual Prático: Construindo o Égua Mano! Gamer do Zero em Go

**Guia Passo a Passo de Engenharia de Software, Game Loop e Baixo Nível**

**Autor & Engenheiro:** Lucivaldo Junior (Luci Junior)

**Linguagem:** Go (Golang 1.22+) | **Engine:** Ebitengine v2 | **Alvo:** Desktop & WebAssembly (WASM)

### Por que este Manual Existe?

Este documento não é um sumário teórico e nem uma lista estática de funções. É um **manual de construção sequencial**. Ele ensina você a reconstruir este jogo (ou qualquer motor 2D em Go) **linha por linha, do zero absoluto até a arquitetura corporativa final**, explicando o que a CPU, a GPU e a memória RAM estão fazendo a cada passo.

### Roteiro da Construção

1. [Capítulo 1: O Módulo Go e a Janela Vazia \(O Loop de 60 FPS\)](#).
2. [Capítulo 2: Física Clássica — Gravidade, Impulso e Chão](#)
3. [Capítulo 3: A Grande Separação — Standard Go Layout \( `cmd/` e `internal/` \)](#)
4. [Capítulo 4: Polimorfismo sem Herança — A Interface `PlayerCharacter`](#)
5. [Capítulo 5: Projéteis e Balística — Tiro de Baladeira e Rugido Felino](#)
6. [Capítulo 6: Inimigos, Colisão AABB e a Mecânica de Pisão \(\*Stomp\*\)](#)
7. [Capítulo 7: Cenários em Camadas e o Efeito Parallax Scrolling](#)

8. [Capítulo 8: Áudio Procedural na Memória RAM e o Padrão Singleton](#)
  9. [Capítulo 9: Compilação Cruzada para WebAssembly e Ponte `syscall/js`](#)
  10. [Capítulo 10: Roteiro de Apresentação e Defesa em Entrevistas Técnicas](#)
- 

## Capítulo 1: O Módulo Go e a Janela Vazia (O Loop de 60 FPS)

---

### 1. Inicializando o Projeto

No terminal Linux, crie a pasta do projeto e inicie o módulo Go:

```
mkdir egua-mano_gamer
cd egua-mano_gamer
go mod init github.com/luci-jr/egua-mano_gamer
go get github.com/hajimehoshi/ebiten/v2
```

### 2. O Mínimo Código Funcional ( `main.go` )

Crie o arquivo `main.go`. No Go, todo jogo Ebitengine precisa implementar a interface implícita `ebiten.Game`:

```
package main

import (
    "image/color"
    "log"

    "github.com/hajimehoshi/ebiten/v2"
)

// Game representa a nossa aplicação e seu estado na memória
type Game struct{}

// Update executa a lógica e a física 60 vezes por segundo (Tick fixo)
func (g *Game) Update() error {
    return nil
}

// Draw renderiza os elementos gráficos na tela a cada frame
func (g *Game) Draw(screen *ebiten.Image) {
    // Pinta a tela de azul céu amazônico
    screen.Fill(color.RGBA{R: 70, G: 130, B: 180, A: 255})
}

// Layout define a resolução lógica interna fixa (retrô), independente da janela real
func (g *Game) Layout(outsideWidth, outsideHeight int) (screenWidth, screenHeight int) {
    return 340, 210 // Proporção clássica 16:10 retrô
}

func main() {
    ebiten.SetWindowSize(680, 420) // Janela abre no dobro do tamanho lógico
    ebiten.SetWindowTitle("Égua Mano! Gamer")
    if err := ebiten.RunGame(&Game{}); err != nil {
        log.Fatal(err)
    }
}
```

### 💡 O que aprendemos aqui:

- **Taxa de Atualização Fixa (Tick Rate):** O método `Update()` roda exatamente a 60 ticks/s (a cada 16.6 milissegundos). A física deve ser calculada aqui, nunca dentro do `Draw()`.
- **Desacoplamento de Resolução ( `Layout` ):** Ao travar a resolução lógica em `340x210`, o jogo pode abrir em um monitor 4K ou num celular antigo: a GPU escala os pixels sem distorcer o cálculo de física.

## 🏃 Capítulo 2: Física Clássica — Gravidade, Impulso e Chão

Para mover um personagem, precisamos de coordenadas cartesianas ( $X$ ,  $Y$ ) e vetores de velocidade.

## 1. Estrutura do Herói

```
type Player struct {
    X, Y          float64
    VelocityY     float64
    IsJumping     bool
    GroundY       float64
}

func NewPlayer(groundY float64) *Player {
    return &Player{
        X:          40.0,
        Y:          groundY,
        GroundY:    groundY,
    }
}
```

## 2. A Matemática do Pulo e da Gravidade

No plano 2D de computadores, o eixo  $Y$  cresce **para baixo** ( $Y=0$  é o topo da tela).

- Para pular: aplicamos uma velocidade negativa (ex:  $-7.0$ ).
- A gravidade: a cada tick somamos uma constante positiva (ex:  $+0.38$ ).

```
const (
    Gravity    = 0.38
    JumpForce  = -7.2
)

func (p *Player) Jump() {
    if !p.IsJumping {
        p.VelocityY = JumpForce
        p.IsJumping = true
    }
}

func (p *Player) Update() {
    // Aplica a gravidade na velocidade vertical
    p.VelocityY += Gravity
    p.Y += p.VelocityY

    // Colisão com o solo (Ground Clamping)
    if p.Y >= p.GroundY {
        p.Y = p.GroundY
        p.VelocityY = 0
        p.IsJumping = false
    }
}
```



## Capítulo 3: A Grande Separação — Standard Go Layout ( `cmd/` e `internal/` )

Quando colocamos inimigos, partículas, cenários e som, manter tudo em um único arquivo cria dependências circulares e impossibilita testes unitários.

Adotamos a estrutura corporativa recomendada pelo Google:

```
egua-mano_gamer/  
├── cmd/  
│   └── runner/  
│       └── main.go      # Ponto de entrada de inicialização puro  
├── internal/           # Protegido pelo compilador do Go!  
│   ├── audio/         # Síntese e drivers de som  
│   ├── entities/      # Personagens, obstáculos, projéteis e relíquias  
│   ├── game/          # Engine central (Game Loop, estados e colisões)  
│   ├── scenery/       # Camadas de fundo e parallax  
│   └── ui/            # HUD, menus, fontes e diálogos  
├── go.mod  
└── go.sum
```

### Por que a pasta `internal/` é fundamental no Go?

- O compilador do Go impõe uma regra rigorosa: **nenhum módulo ou pacote fora deste repositório consegue importar nada que esteja dentro de `internal/`**.
- Isso blinda o núcleo do seu jogo contra acoplamentos externos indesejados e garante encapsulamento corporativo real.



## Capítulo 4: Polimorfismo sem Herança — A Interface `PlayerCharacter`

### 1. O Problema

Queremos dois personagens jogáveis:

1. **O Garoto Curumim:** Rápido, pulo duplo humano, atira carvão de açaí com baladeira.
2. **A Onça-Pintada:** Salto elástico de predador, rastejo felino, dispara ondas de rugido sônico.

Como em Go não existe `class Garoto extends Hero`, como a Engine central controla qualquer um dos dois sem duplicar código?

### 2. A Solução: Interface Idiomática

Criamos o contrato em `internal/entities/player.go`:

```
package entities

import "github.com/hajimehoshi/ebiten/v2"

type PlayerCharacter interface {
    MoveForward(speed float64)
    MoveBackward(speed float64)
    StopRunning()
    Jump() (jumped bool, isDouble bool)
    ReleaseJump()
    SetCrouch(crouch bool)
    TriggerAttack()
    GetShootOrigin(groundY float64) (x, y, vx, vy float64)
    GetBounds(groundY float64) (x, y, w, h float64)
    Bounce(strength float64)
    SetStarPower(timer int)
    GetStarPower() int
    Update()
    Draw(screen *ebiten.Image, groundY float64, ticks int, invincibleTicks int)
    GetName() string
    GetHeroKind() int
}
```

### 3. Implementação Concreta

Tanto `Onca` quanto `Garoto` implementam todos esses métodos. Na struct `Engine` em `internal/game/game.go`, declaramos apenas:

```
type Engine struct {
    player entities.PlayerCharacter // Pode ser *Onca ou *Garoto!
    // ...
}
```

Quando o jogador aperta espaço:

```
e.player.Jump()
```

O compilador resolve a chamada via *vtable* (tabela virtual) em tempo de execução com altíssima performance.



## Capítulo 5: Projéteis e Balística — Tiro de Baladeira e Rugido Felino

Em `internal/entities/projectile.go`, gerenciamos a física de projéteis.

## 1. O Ciclo de Vida do Projétil

- Um projétil nasce na mão do Garoto (ou boca da Onça) via `GetShootOrigin()`.
- Possui posição  $(X, Y)$  e velocidade  $(V_x, V_y)$ .
- Move-se a cada tick:  $X = X + V_x$ .
- **Descarte de Memória:** Quando  $X > \text{ScreenWidth}$ , o projétil é removido da slice para não causar vazamento de memória (*memory leak*).

```
func (pm *ProjectileManager) Update(screenWidth float64) {
    active := pm.projectiles[:0] // Reutiliza a fatia de memória (Zero Allocation)
    for _, p := range pm.projectiles {
        p.X += p.VX
        p.Y += p.VY
        if p.X < screenWidth && p.Active {
            active = append(active, p)
        }
    }
    pm.projectiles = active
}
```

## Capítulo 6: Inimigos, Colisão AABB e a Mecânica de Pisão (*Stomp*)

### 1. Detecção de Colisão AABB (Axis-Aligned Bounding Box)

Dois retângulos colidem se, e somente se, houver sobreposição nos dois eixos simultaneamente:

$$\text{Colisão} = (A.x < B.x + B.w) \wedge (A.x + A.w > B.x) \wedge (A.y < B.y + B.h) \wedge (A.y + A.h > B.y)$$

### 2. A Mecânica do Pisão (*Stomp* estilo Super Mario)

O segredo está em separar a colisão frontal do impacto descendente:

```
func (m *ObstacleManager) CheckStomp(playerX, playerY, playerW, playerH, playerVY, groundY float64) (bool, float64, float64, string) {
    playerBottom := playerY + playerH

    for _, obs := range m.obstacles {
        if !obs.Active || obs.Deleted {
            continue
        }

        obsX, obsY, obsW, obsH := obs.GetBounds(groundY)
        obsTop := obsY + 4.0 // Zona de tolerância da cabeça

        // Checa se o jogador atingiu horizontalmente o inimigo
        horizontalOverlap := (playerX+playerW*0.8 > obsX) && (playerX+playerW*0.2 < obsX+obsW*0.8)

        // Pisão ocorre se os pés do herói estão caindo sobre o topo da cabeça
        if horizontalOverlap && playerBottom >= obsY-2.0 && playerBottom <= obsTop+1.0 {
            obs.Deleted = true // Inimigo esmagado!
            return true, obsX + obsW/2, obsY, obs.Type
        }
    }
    return false, 0, 0, TypeGround
}
```

Ao confirmar o pisão:

1. O inimigo é marcado como `Deleted`.
2. A `Engine` toca o som de vitória e adiciona \$+150\$ pontos.
3. A `Engine` chama `player.Bounce(-6.2)` — aplicando impulso elástico ascendente imediato!
4. **Salvaguarda de Pulo:** `CheckCollision()` ignora o dano se o inimigo já foi marcado como derrotado no mesmo tick.

## Capítulo 7: Cenários em Camadas e o Efeito Parallax Scrolling

O segredo da ilusão de profundidade 2D é o **Parallax**: objetos distantes movem-se mais devagar que objetos próximos.

Em `internal/scenery/ver_o_peso.go`:

- **Camada do Céu e Nuvens:** Desloca-se a  $0.1 \times \text{velocidade}$ .
- **Camada das Cúpulas do Ver-o-Peso / Docas:** Desloca-se a  $0.4 \times \text{velocidade}$ .
- **Camada dos Barcos e Baía:** Desloca-se a  $0.7 \times \text{velocidade}$ .
- **Solo dos Trilhos:** Desloca-se a  $1.0 \times \text{velocidade}$  (velocidade real de corrida).

```
func (b *Background) Update(speed float64) {
    b.skyX -= speed * 0.1
    b.buildingsX -= speed * 0.4
    b.groundX -= speed * 1.0

    // Loop contínuo: reseta o offset quando ultrapassa a largura da imagem
    if b.buildingsX <= -b.imageWidth {
        b.buildingsX = 0
    }
}
```

## 🎵 Capítulo 8: Áudio Procedural na Memória RAM e o Padrão Singleton

### 1. Síntese Matemática Sem Arquivos Externos

Em vez de carregar arquivos MP3 pesados pela rede, sintetizamos o áudio como sinais PCM de 16-bit a 44.100 Hz diretamente na CPU.

- **Onda Senoidal:** Som puro e limpo (flautas, assovios):  $S(t) = \sin(2\pi \cdot f \cdot t)$
- **Onda Quadrada:** Timbre retrô característico de Chiptune 8-bit (NES/GameBoy).
- **Ruído Branco (White Noise):** Valores aleatórios para sintetizar o chimbal e as maracas do Carimbó:  $\text{Noise} = \text{rand}(-1.0, 1.0)$

### 2. O Padrão Singleton no Driver de Áudio

O Ebitengine não permite abrir dois contextos de áudio concorrentes no sistema operacional. Para evitar `panic` ao rodar testes simultâneos com `go test ./...`, usamos o padrão Singleton:

```
var (
    audioCtx      *ebitenaudio.Context
    audioCtxOnce sync.Once
)

func getAudioContext() *ebitenaudio.Context {
    audioCtxOnce.Do(func() {
        audioCtx = ebitenaudio.NewContext(44100)
    })
    return audioCtx
}
```

O `sync.Once` da biblioteca padrão do Go garante que a inicialização execute exatamente uma única vez, sendo imune a concorrência (*thread-safe*).



## Capítulo 9: Compilação Cruzada para WebAssembly e Ponte `syscall/js`

### 1. O Desafio Mobile no Browser

Navegadores em smartphones ignoram eventos sintéticos de teclado ( `KeyboardEvent` ) disparados por botões na tela por motivos de segurança ( `isTrusted: false` ).

### 2. A Solução com Build Tags do Go

Dividimos o código de leitura de inputs em dois arquivos:

1. `internal/game/input_js.go` :

```
//go:build js && wasm

package game

import "syscall/js"

func getVirtualKey(keyName string) bool {
    v := js.Global().Get("window").Get("_virtualKeys").Get(keyName)
    return v.Truthy() && v.Bool()
}
```

2. `internal/game/input_other.go` :

```
//go:build !(js && wasm)

package game

func getVirtualKey(keyName string) bool {
    return false // Desktop nativo lê teclado físico direto
}
```

### 3. Compilando para WebAssembly

```
GOOS=js GOARCH=wasm go build -o docs/game.wasm ./cmd/runner
```

Esse comando gera um arquivo binário WebAssembly puro que roda diretamente no navegador a 60 FPS estáveis com aceleração WebGL!

## Capítulo 10: Roteiro de Apresentação e Defesa em Entrevistas Técnicas

Quando um recrutador ou tech lead perguntar sobre este projeto, conduza a explicação com esta narrativa estruturada:

### 1. Concepção:

- "Eu escolhi construir o Égua Mano! Gamer em Go para dominar a linguagem em cenários reais de alto desempenho e baixo consumo de memória, fugindo do óbvio dos tutoriais básicos de CRUD."

### 2. Arquitetura:

- "Estruturei o projeto seguindo o Standard Go Project Layout, isolando o domínio de entidades, física e áudio dentro da pasta `internal/` para garantir blindagem pelo compilador do Go."

### 3. Padrões de Projeto & Boas Práticas:

- "Substituí a herança tradicional por polimorfismo estrito através da interface `PlayerCharacter`, permitindo alternar personagens com comportamentos e armas totalmente distintas sem alterar uma linha da Engine central."
- "Resolvi concorrência no driver de som aplicando o padrão Singleton com `sync.Once`."

### 4. Física & Performance:

- "Implementei cálculo vetorial de colisão AABB desacoplado em 60 ticks por segundo, criando mecânica de pisão (Stomp) com impulso elástico e detecção matemática de caixas sobrepostas."

### 5. Multiplataforma:

- "Compilei o mesmo código-fonte para Desktop nativo e WebAssembly, utilizando Build Tags e ponte de memória de baixo nível via `syscall/js` para compatibilidade total com telas de celular e navegadores."

**Conclusão:** Seguindo este roteiro, você não é apenas um usuário de bibliotecas; você é um engenheiro capaz de projetar sistemas de alto desempenho, entender ciclo de vida de memória e explicar com autoridade cada decisão técnica tomada.